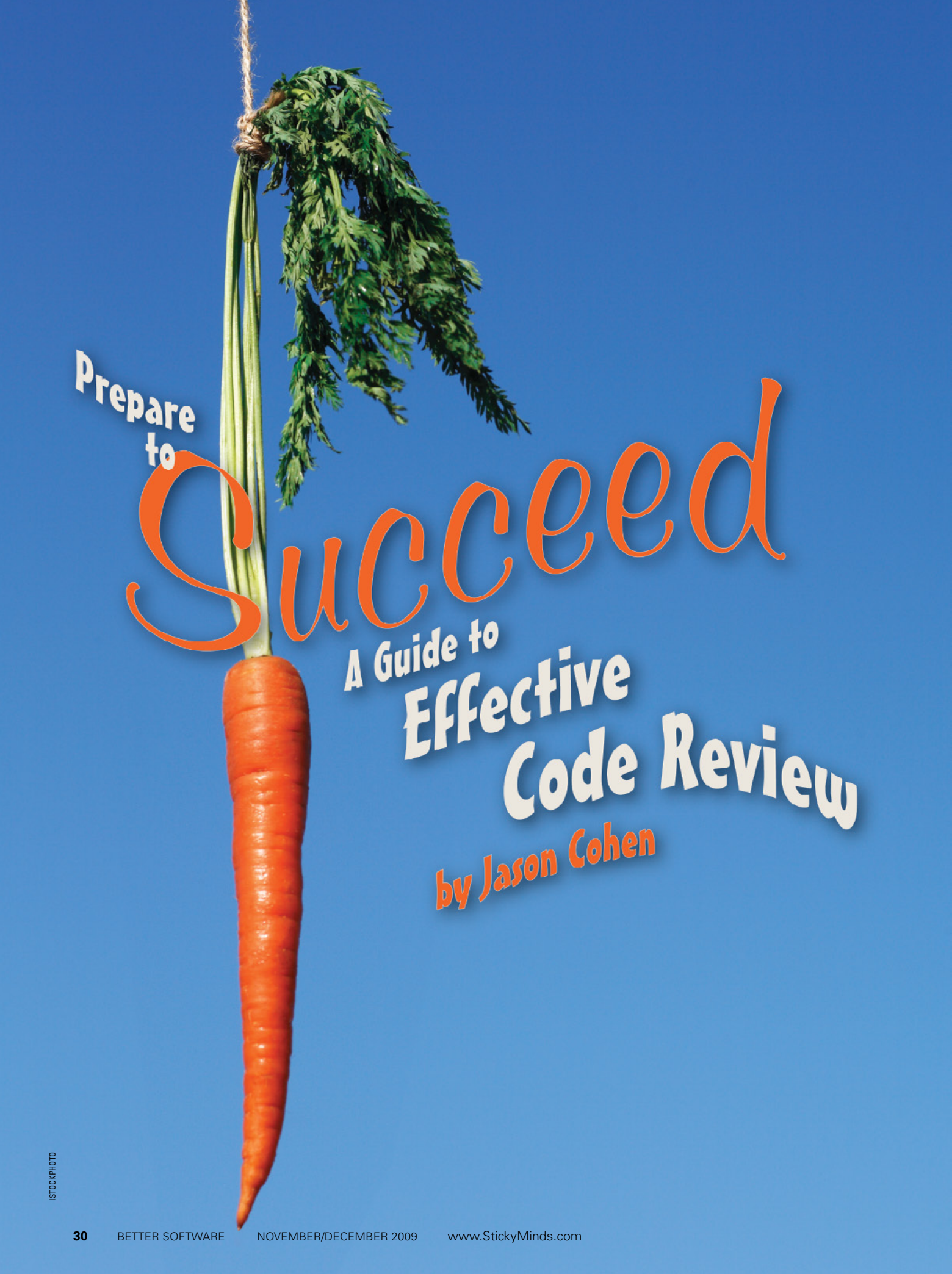


Prepare to Succeed

A Guide to Effective Code Review

by Jason Cohen

ISTOCKPHOTO

Most people despise doing code reviews. Reviews take time away from the things programmers enjoy, such as crafting code. As it is, no one has enough time to keep up with demanding release schedules. Worse, code reviews are actual invitations for criticism and provide an ideal opportunity to showcase to the world all of the dumb mistakes you've made.

At the same time, most programmers will admit that if someone else were to look over their code, bugs would be found. And, even though most programmers hate reviews, the concept makes sense if you think it through. Two brains are better than one. You wouldn't write a book or even an article without having someone else—usually several people—review it. Extra eyeballs catch things even the most diligent author overlooks. Whether code or prose, I sometimes review my own writing five times and still miss errors that another person catches immediately—not because I'm a moron, but because I wrote it and I'm too close to it to see the problems.

Typos and clumsy sentences in an article won't crash a system or cause data loss, but if I'm writing code, the stakes are much higher. An error in my code could cause a catastrophe, so that makes another pair of eyes on the code I write even more important.

"OK," you say. "Maybe we *should* be doing code reviews. But how do we do them efficiently, so they're worthwhile? How do we make them not painful? How do we position our code reviews for success and actually get people to do them?"

I've spent a lot of time researching and doing code reviews and working with teams that do them. And, while code review processes and metrics are fine in theory, successfully implementing code review for a team is an art. To get started and position the endeavor for success, I've found the following approach works best.

Start Small

The best way to start doing code reviews is at a small scale, on only a handful of files. Don't plan to review all of your code at the outset. Instead, use the following suggestions for limiting review scope to help you realize the most useful and persuasive results. Choose as many of them as make sense for your team:

- Review modifications to the core module that is referenced by all other code.
- Only review changes to the stable branch.
- Don't review entire files; just review the changes (code review tools or version control systems can highlight them for you).
- Let the developers choose the top ten most risky files to review. Also let them decide when else a review is necessary, such as when they know a domain expert can lend assistance or when they're feeling "iffy" about a particular code change.
- Review only unit tests (after all, they're source code files and can be reviewed like everything else). If these are complete and well written, you can safely fix bugs or refactor later.

Motivate Through Your Own Real Data

Now let's address the most difficult question: How do you motivate people to do code reviews? Everyone wants to improve code quality, but do they really believe code review is an efficient way to do that? A simple way to get buy-in (and subsequent participation) is to demonstrate the value of code reviews to your team on its specific projects.

One easy way to start is by sharing this factoid I've found in my research, gleaned from stats I collected working with thousands of programmers: When code reviews are conducted efficiently, the average team finds and fixes a bug every ten to fifteen minutes.

That's pretty powerful. In fact, there's no other process in software development that identifies and fixes bugs that quickly.

"But will it really save us that much?" Good question. The easiest way to find out is to try it. First, get your team members to agree to review code for one week, spending a total of two to two-and-a-half hours each. Then, meet with team members to establish the process to use (described in the Ways to Review

Ways to Review Code

You have a choice of multiple methods to review code. Pick the one that works best for your team.

Formal meetings—Teams print out code and review it together in meetings. The traditional method of code review, formal meetings are eschewed by many because they are usually excruciating and too time consuming to be practical.

Pair programming—This is sometimes the code review method of choice for agile developers. Programmers write all code as a two-person team and can catch each other's mistakes.

Walkthrough/over the shoulder—This informal approach works well when developers are in the same office. The author steps through his or her code with the reviewer.

Email pass-around—This informal method works pretty well for geographically distributed teams. They can email each other code as it's developed and send comments back and forth; however, they still have to manually attach files and reference line numbers.

Tool-assisted—The most efficient method, tool-assisted code reviews use dedicated software to automate uploads, facilitate online discussions (sometimes with comments directly on the code itself, so programmers don't have to search for line numbers), and keep the process moving along. Tool-assisted reviews take about one-fifth of the time of formal reviews because the tools handle the tedious parts. They work well for both local and distributed teams.

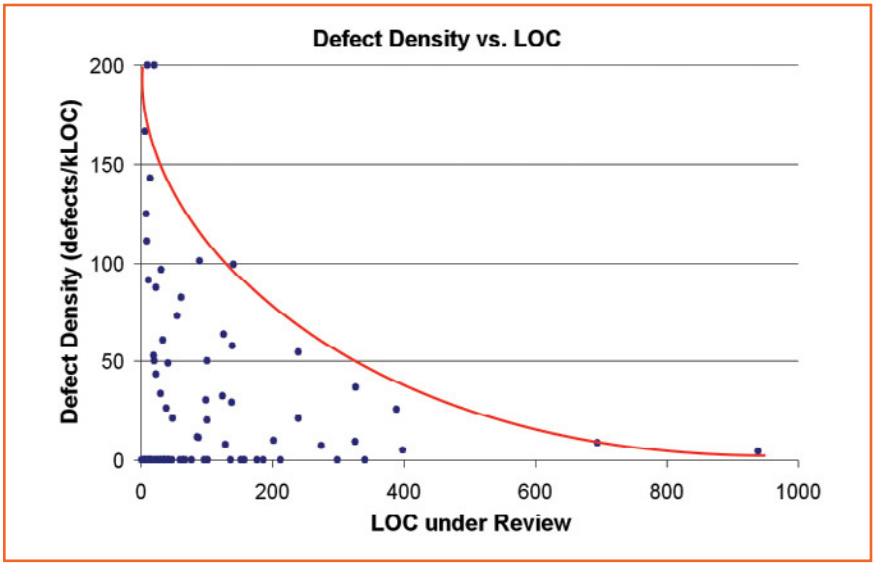


Figure 1: Defect density dramatically decreases when the number of lines of inspection goes above 200 and is almost zero after 400.

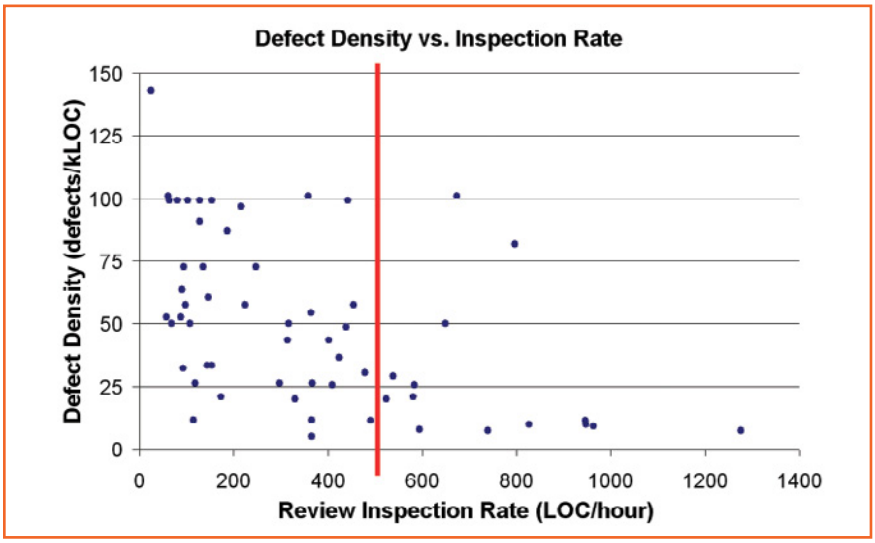


Figure 2: Inspection effectiveness falls off when greater than 500 lines of code are under review.

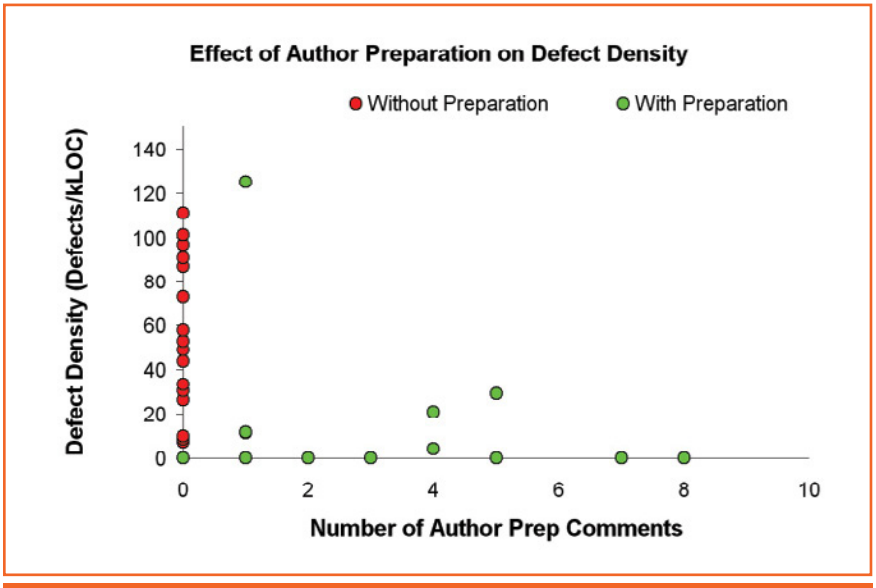


Figure 3: The striking effect of author preparation on defect density.

Code sidebar). For this evaluation, I suggest tool-assisted, over-the-shoulder, or email pass-around. These methods are easiest to conduct and track.

Have everyone do reviews for twenty to thirty minutes per day for that one week and capture two simple metrics: number of bugs found and time spent. Calculate the rate at which your team finds bugs by dividing the total time spent on reviews by the number of defects found. For example, twenty hours spent divided by eighty defects found yields fifteen minutes spent to find one defect.

Now get the team together, discuss the results, and have the group decide if it was worth it.

Even the most crotchety of programmers can suffer through this process for a week, and then you'll have hard data that shows whether code reviews are worthwhile in your organization. If they don't buy in to the value of code reviews at this point, it's hard to force them. If they do agree that code reviews are worthwhile for improving software quality, you'll almost certainly have successful reviews going forward. Now slowly expand the scope of the reviews.

Note that during this week you'll need to take the time to collect metrics like defects found and time spent. But after that first week, if you don't care about tracking this information, then don't. In that case, future code review may be even less effort than it was when you tried it out.

Get Maximal Results, Spend Minimal Time

"All right," you agree. "Code review is clearly worthwhile, so we'll try it. But how do we get the best results in the least time?"

First, choose a style of code review that is lightweight and easy but tracks things like number of defects found and time spent in review, so you can evaluate your results. Any of the methods listed in the sidebar will work—except formal code reviews, which are not lightweight and take too much time—though some require more manual work to collect metrics. For a good compromise, use an evaluation copy of a commercial peer code review tool or an open source tool. Then,

you can enjoy the benefit of structured guidance without affecting your budget.

Once you've decided on the process you're going to use and which files to review, follow the guidelines below to make the most of your code reviews. They're based on a code review study at Cisco Systems that encompassed fifty programmers, 2,500 code reviews, and 3.2 million lines of code (see the StickyNotes for more information).

- Review no more than 200 to 400 lines of code (LOC) at one sitting. Outside of that range, your ability to find defects plummets, as shown in figure 1.
- Review fewer than 300 to 500 LOC per hour. It stands to reason that if you don't spend much time on the review, you won't find many of the bugs that lurk there. But if you try to review too much code at once, you won't be able to devote enough detailed attention to the code you're reviewing, as shown in figure 2.
- Limit reviews to sixty to ninety minutes. After that point, you get tired, lose focus, and your performance starts to drop off. Note that this number is about the same for other focused activities.
- Have the author annotate the code before the review starts. This practice provides two huge benefits. First (and most obvious), the annotations help the reviewer know which files to look at and explain the reasoning behind methods chosen. The second benefit is even more powerful. Often, as authors are rethinking and explaining their code, they discover their own defects even before the review begins, as shown in figure 3. This kind of self-review is a good idea even if your team doesn't do code reviews!
- Use checklists. They greatly improve results for reviewers and authors, alike. Checklists remind programmers to pay attention to easy-to-overlook tasks, such as proper error checking, unit tests, invalid value checking, and others. Personal checklists also can be a big help. Figure out what mistakes you typically make and keep a checklist of the five to ten most common ones. Once you stop making those mistakes, update the list to cover whatever mistakes you now perpetrate. See the StickyNotes for more detailed information on how to build checklists.
- Make sure that bugs are fixed. It sounds simple, but since bugs found during review aren't automatically tracked, you need to put a process in place to ensure that they're dealt with.
- Encourage a positive culture around reviews. Make sure everyone understands that finding bugs is a good thing. Each defect found is a bug that gets fixed at the optimal time in the cycle and never reaches the customer. The results of code review (how many bugs each programmer introduces) should never be used as the basis for a programmer's performance evaluation (except in cases where it shows that the person is a strong team player who conducts a lot of code reviews). I'll give specific tips on how to set the right tone for reviews in the next section.
- Always review some code. Even if you don't have time to review all of it, you'll find that programmers become more careful if they know that someone else might be checking their work. They'll check it themselves first, because they want to avoid public embarrassment and want potential reviewers to see the brilliance that is their work. We call this the Ego Effect, and it kicks in if a team reviews at least 20 to 33 percent of a project's code.

Position Code Reviews for Success: Suggestions for Managers

The most important thing a manager can do to ensure successful code reviews is to set the right tone and help programmers adopt the right attitude toward reviews.

Why Code Review?

- Improve software quality—You won't just ship fewer bugs, you'll improve design, structure, documentation, maintainability, testing, and even code comments.
- Help your team communicate better—Code reviews get people talking, collaborating, and building trust in addition to better code. Once the "ice is broken," many teams report vast improvement in how they work together.
- Mentor and teach new team members—It's a good way to let them work on "real" projects while making sure they don't do any serious damage, and it's a fantastic way for them to learn.
- Disseminate knowledge throughout the team—Code reviews deepen understanding about how your code works and spread best practices. It's a great way to make sure that more than one or two people truly understand your code.

Before you kick off the review process, allow team members to air their concerns up front so that the process addresses these issues from the start. Open lines of communication are crucial. It helps if team members already get along, but if they don't, and you do code reviews with the right attitude, you might find they'll start to get along. It also helps if team members have the ability—and feel safe enough—to laugh at themselves and know that others support and respect them even when they do something shockingly bone-headed (which we all do at least once in a while).

You can use these points to set the stage and define the right "code review culture" at the outset:

- **Finding defects is good—**You want to find defects in code reviews. Every bug found now is one that is not found later in test or by customers. It's a bug that's cheaper and easier to fix. Finding bugs in code reviews should be applauded.
- **Code review is about the code, not the author of the code—**It's about giving suggestions to improve the code; it should not be a personal evaluation or an affront to the coder. Don't make it personal.

Find Bugs Early, Save Exponentially

When folks catch bugs early, the error can be easily rectified without major rework. Consider that fixing bugs found during test costs about eight to twelve times more to fix than bugs found and fixed when the software is still in development. The tester has to discover an error, document it, and send the code back to development. Then the programmers search until they find the bug, and they start to fix it—which might be many months after the code was written, when the error has affected many other things. To compound the problem, the code is no longer fresh in the programmers' minds, or no one even remembers who wrote that section of code.

Once the software is released into customers' hands, the cost to find and fix bugs rises to thirty to one hundred times what it costs in the development stage (and that's not counting damage to your company's reputation). So, if code reviews truly help you find a bug every ten to fifteen minutes, that's a huge cost savings and trouble avoidance for everyone.

Share this information with your team, and you'll likely get a lot more buy in. Even though most teams barely have time to meet their tight development schedules, code reviews (done right) save time in the long run.

- **Don't tolerate bullying**—Occasionally, a team member uses code review as an opportunity to destroy the self-confidence of another programmer or to try to establish intellectual superiority over all. It's the manager's job to ensure that reviews are conducted in a safe and encouraging environment. If a team member starts to "abuse her power," it's time to have a chat to remind her about the right spirit for code reviews.
- **Work with everyone on the team, not just your buddies**—Team members will learn more if they make the rounds and review everyone's code. This way they can get to know each other and pick up new ideas from more people.
- **Code quality is a team effort**—Code reviews strongly support this goal, and they help build teams. But, if someone on your team isn't following the rules (such as by bullying), it's best not to single her out in front of the team; that approach likely will cause even more trouble. Instead, remind the team as a whole that finding defects is a good thing, and that the number of bugs in someone's code does not correlate to her innate intelligence. After all, harder code and newer code have more bugs, regardless of the author.
- **Inject humor and have fun at every opportunity**—Code review can be the perfect opportunity for team members to joke around and to make light of themselves and their mistakes. If you encourage humor, then everyone becomes more comfortable laughing together—and laughing at themselves. And, they probably won't be offended by review comments or despise the time they spend reviewing. With regular humor and fun, overall morale is likely to improve substantially.

Position Code Reviews for Success: Tips for Programmers

While it's easy to look down on errors and disdain those who make them, remember that everyone—even you—makes mistakes. Code review is about recognizing and supporting a phenomenon that is inherently human. To keep the spirits up and attitudes positive, programmers can use these simple guidelines:

- **Always find something good to say**—Better yet, lead with that. In general, offer as much praise as you can.
- **Keep in mind that you're working to improve the code, not to criticize the author**—This mindset helps you frame comments in a more positive, less personal way and helps keep the author focused on the goal of improving code quality, as well.
- **Be patient and respectful with everyone**—Although it can be difficult, this tenet applies especially to colleagues who are less experienced, knowledgeable, or skilled than you. Your encouragement and equanimity will pay off in the end—for you, for them, and for the whole team.
- **Embrace the opportunity to learn from others**—It's not often you get the chance for one-on-one, targeted instruction. You can learn something from everyone, even the greenest member of your team.
- **When disagreements happen, win and lose with grace**—Especially win. You'll foster future allies rather than enemies. Added bonus: You won't look like a jerk.
- **Ask questions rather than making statements**—Asking someone what her logic was when he did something is a lot less offensive than saying, "This is wrong." Especially because it might not be wrong, and her answer might teach you a thing or two. Asking a question sets the stage for an evenhanded discussion rather than a one-sided criticism.
- **Give specific, timely feedback**—It's helpful and time-saving for all if you tell the author exactly where a problem lies (exact line number) and give her enough details to be able to fix the problem in a satisfactory way. Obviously,

once the author has submitted her code to you for review, she's waiting to get it back. Be considerate and don't make her wait too long.

- **Be thankful**—Someone is taking her valuable time to help you produce a better product, share her knowledge, and help your team in general. Appreciation is in order.

Get Started!

You already know that code review will improve all aspects of your code—from design to structure, maintainability to documentation, testing to general quality—or you wouldn't have read this far. If you're still skeptical about the team dynamics, I'll proffer a few final insights. I've worked with hundreds of teams that implemented code review, and they repeatedly report social benefits that surprised them.

First, everyone's happy and excited about producing a better product and about learning new things. The goodness continues to grow as code quality improves and as team members pick up more and more new tricks from their peers.

And although code reviews may be conducted across oceans, teams find that when they talk to their teammates around a mutual goal, they start to build their relationships. Developers come out of their caves and start to work together, rather than in parallel. Now that they have something to talk about, they have the opportunity to bond with others in person, by email, in meetings, or online. Teams with the right code review culture report that everyone is having more fun and getting along better.

So, what are you waiting for? Try your hand at code review for a week. Social benefits aside, if you do the math, you know you can't afford not to. {end}

Sticky Notes

For more on the following topics go to www.StickyMinds.com/bettersoftware.

- Code review study
- How to build checklists

The **Seven** Habits of Highly Effective Testing Organizations

A New White Paper by

Testing Expert Lee Copeland

How are you tackling quality issues?

Is your testing organization regarded as trusted advisor – or a perceived bottleneck to getting releases out the door?

From this white paper you will learn:

. How Agile is changing the way we test

. Why the testing organization no longer needs to be a distinct group

. The importance of risk-based testing and its main elements

. Change management's importance for producing quality software

Lee Copeland brings us a look at the habits that we should adopt to take testing to a new level and reap greater value from QA efforts.

Get the free white paper, compliments of MKS:
www.mks.com/seven_habits



Contact us: 1 800 613 7535 | info@mks.com

MKS